



PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

STUDENT & COLLEGE MANAGEMENT SYSTEM (SCMS)

Program	B.Tech CSE – AIML
Section	Rhinos
Faculty	Prof.Sahithi
Date of Submission	10-10-2026
Academic Year	2026 – 27

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0104029	Vardan Desai	____7990150741____ _____	____vardandesai2029@gmail.com____ _____

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course. The project provided an opportunity to translate relational database concepts into a working Python application connected to MySQL.

I thank my course faculty, _____Prof.Sahithi_____, for the guidance and feedback provided during the design and development of the system. The work helped strengthen my understanding of ER modelling, relational schema design, normalization, SQL constraints, triggers, stored procedures, views, transactions and database application development.

I also acknowledge the official MySQL and Streamlit documentation used to verify implementation concepts and application behavior. The submitted prototype and report are intended for academic evaluation and demonstrate the practical integration of a relational DBMS with a small Python application.

UNDERTAKING

I hereby declare that the project entitled “Student & College Management System (SCMS)”, submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual academic work carried out under the guidance of the course faculty.

I undertake that the database design, SQL scripts, Python application, results and supporting materials included in the submission have been prepared for academic purposes. Wherever external documentation or technical references were consulted, they have been acknowledged in the references section.

I further undertake that I am responsible for the correctness and authenticity of the material submitted and that the report reflects the current Python, Streamlit and MySQL implementation of the project.

Signature: _____ Vardan _____

Date: _____ 10-10-2026 _____

Place: _____ Woxsen _____

Name: Vardan Desai

Roll Number: 25WU0104029

Program: B.Tech CSE – AIML

TABLE OF CONTENTS

Abstract	8
List of Tables	5
List of Figures	6
List of Acronyms	7
1. Problem Statement, Solution Strategy and ER Diagram	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	10
1.7 Entity and Relationship Summary	11
2. Logical Database Design	12
2.1 Relational Schema	12
2.2 Keys and Integrity Constraints	12
2.3 Functional Dependencies	13
2.4 Normalization up to Third Normal Form (3NF)	14
2.5 Data Dictionary	15
2.6 Implementation of Logical Design: DDL and Sample Data	19
3. Application Development Using Logical Database Design	20
3.1 Application Overview	20
3.2 Development Environment and Database Connectivity	20
3.3 Application Modules and Database Mapping	20
3.4 CRUD Operations	21
3.5 Business Rules, Validation and Error Handling	21
3.6 Reports and Database Operations	21
3.7 Application Screens	22
3.8 Testing and Results	26
4. Conclusion	27
4.1 Limitations and Future Enhancements	27
5. References	28
6. Appendices	29

LIST OF TABLES

Table 1. Scope and Assumptions	9
Table 2. Proposed Solution Strategy	10
Table 3. Functional Requirements	10
Table 4. Entity and Relationship Summary	11
Table 5. Relational Schema	12
Table 6. Keys and Integrity Constraints	12
Table 7. Normalization Status up to 3NF	14
Table 8. Data Dictionary	15–19
Table 9. Database Implementation Environment	19
Table 10. Application Development Environment	20
Table 11. Application Module-to-Database Mapping	20
Table 12. CRUD Operations	21
Table 13. Business Rules, Validation and Error Handling	21
Table 14. Reports and Database Operations	21
Table 15. Test Cases	26

LIST OF FIGURES

Figure 1. Entity-Relationship Diagram of the Proposed System	11
Figure 2. Login Screen	22
Figure 3. Dashboard and Live Database Metrics	22
Figure 4. Registration / Trigger Validation Screen	23
Figure 5. Examination and Automatic Grade Screen	24
Figure 6. Fee and SQL Reports Screen	25

LIST OF ACRONYMS

DBMS — Database Management System
ERD — Entity-Relationship Diagram
CRUD — Create, Read, Update, Delete
DDL — Data Definition Language
DML — Data Manipulation Language
SQL — Structured Query Language
PK — Primary Key
FK — Foreign Key
3NF — Third Normal Form
ACID — Atomicity, Consistency, Isolation, Durability
UI — User Interface

ABSTRACT

The Student & College Management System (SCMS) is a database-driven academic management prototype developed for a Database Management Systems Project-Based Learning activity. The project addresses the problem of maintaining interdependent college records such as students, programmes, courses, sections, registrations, attendance, examinations, grades, fee bills and payments without duplicating or weakening relationships between them. The main objective is to design and implement a normalized relational database and connect it to a usable Python application. The solution uses MySQL 8.0+ for relational storage and database-side integrity, including primary and foreign keys, UNIQUE, NOT NULL and CHECK constraints, triggers, stored procedures and analytical SQL views. A Streamlit application provides authentication, data-entry modules, CRUD operations, database-rule demonstrations and reporting screens. Fictional seed data is supplied to exercise the schema and reporting logic. The project demonstrates how database-level validation can reject invalid registrations, future attendance records and overpayments while automatically recalculating fee status and examination grades. The resulting prototype provides a compact academic example of ER modelling, normalization, SQL implementation and application-database integration. Thirteen functional test cases executed against the implementation, covering authentication, atomic admission, trigger-based rejections, automatic grading and fee-status recalculation, all produced the expected result.

Keywords: Student Management System, MySQL, Streamlit, SQL Triggers, Normalization, Database Application

CHAPTER 1

PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

A college management workflow produces related records at many points: student admission, programme allocation, course offerings, faculty assignment, registration, attendance, examinations, grades, fee billing and payments. When these records are managed in separate files or disconnected modules, the same facts may be entered repeatedly and the relationships between records become harder to enforce and query.

The project addresses this problem by designing a single relational database that keeps academic and financial entities connected through primary and foreign keys. The final application provides a small interface for working with the database, while MySQL enforces important integrity rules through constraints, triggers and stored procedures. This approach is consistent with the role of a DBMS as the system that maintains relationships and data validity rather than acting only as passive storage [1], [2].

1.2 Objectives

- Design a normalized relational schema for the major student and college records.
- Implement primary keys, foreign keys, UNIQUE, NOT NULL, CHECK and DEFAULT constraints.
- Develop a Python Streamlit application that reads and writes the same MySQL database.
- Automate important validation, fee-status and grade-calculation rules at the database layer.
- Generate academic and financial reports using SQL views and relational queries.

1.3 Scope and Assumptions

Table 1. Scope and Assumptions

Item	Description
In Scope	Student admission; guardian records; departments; programmes; faculty; courses; semesters; sections; registrations; attendance; examinations; grades; fee bills; payments; login accounts; SQL reports.
Out of Scope / Exclusions	Hostel, library, payroll/HR, alumni, timetable automation, online examination proctoring and integration with live university ERP systems.
Assumptions	The project uses fictional seed data for demonstration. MySQL 8.0+ is available; database credentials are supplied through .env; the application is operated by authorized

Item	Description
	users for academic demonstration.
Evidence Basis	Screenshots (Figures 2-6) and test results (Table 15, Appendix C) were captured from the running Streamlit + MySQL prototype using the repository seed data; no results were simulated.

1.4 Proposed Solution Strategy

Table 2. Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Admin, Faculty and Student demonstration roles stored in user_account; role is displayed in the authenticated session.
Major Application Modules	Login, Dashboard, Students, Departments, Programmes, Faculty, Courses, Semesters, Sections, Registration, Attendance, Examinations, Fees and Reports.
Proposed Workflow	User login → Streamlit page → parameterized SQL / stored procedure → MySQL constraints / trigger logic → commit or rejection → updated screen / report.
Important Business Rules	Active students only for registration; duplicate course-semester registration rejected; section capacity enforced; future attendance rejected; dropped-registration attendance rejected; payment over balance rejected; fee status recalculated; grades generated automatically from exam marks.
Expected Reports / Outputs	Section occupancy, attendance summary, examination details, result analysis, academic history, fee dues, student dues and department summary.

1.5 Functional Requirements

Table 3. Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	Admit a student with programme information and optional guardian details.	High
FR-02	Maintain departments, programmes, faculty, courses, semesters and sections.	High
FR-03	Register active students in course sections while preventing duplicates and capacity breaches.	High
FR-04	Record attendance only for valid, non-dropped registrations and reject future dates.	High
FR-05	Record examination marks and generate percentage, letter grade and grade point automatically.	High
FR-06	Issue fee bills, record payments and prevent payment amounts above outstanding balance.	High
FR-07	Generate database-backed reports for academic, attendance, examination and financial analysis.	Medium
FR-08	Authenticate users with bcrypt password hashes and maintain a login session in Streamlit.	Medium

1.6 Entity-Relationship Diagram (ERD)

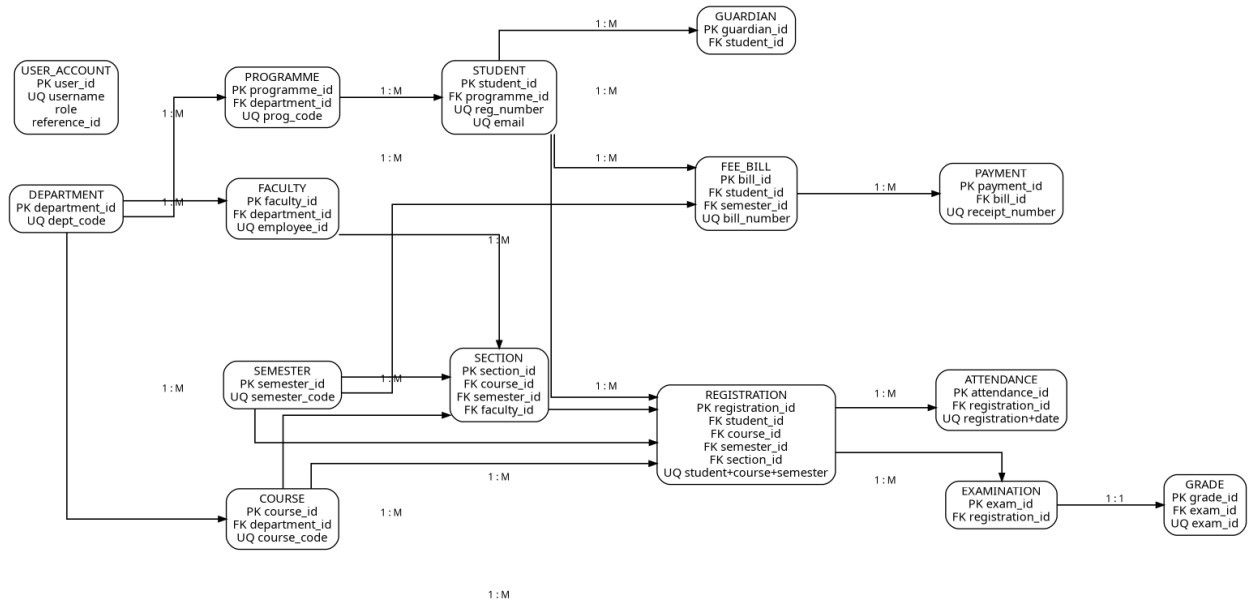


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity / Relationship	Cardinality	Purpose / Rule
Department → Programme	1:M	A department offers multiple programmes; programme references department.
Department → Faculty	1:M	Faculty appointments reference a department.
Department → Course	1:M	Courses are maintained under an academic department.
Programme → Student	1:M	Each student references one programme.
Student → Guardian	1:M	Guardian records are linked to students and cascade on student deletion.
Course + Semester → Section	1:M	Sections are scheduled course offerings for a semester.
Section → Registration	1:M	Registration links students to course sections.
Registration → Attendance	1:M	Each attendance record references one registration.
Registration → Examination	1:M	Each exam record references one student-course registration.
Examination → Grade	1:1	A UNIQUE exam_id in grade supports one calculated grade row per exam.
Student + Semester → Fee Bill	1:M	A student can have multiple fee bills in a semester.
Fee Bill → Payment	1:M	A bill can be settled through multiple payment transactions.

CHAPTER 2

LOGICAL DATABASE DESIGN

2.1 Relational Schema

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	department	Academic department master
2	programme	Degree programme master
3	faculty	Faculty directory and appointments
4	course	Course catalogue
5	semester	Academic term master
6	section	Course-section scheduling
7	student	Student master records
8	guardian	Student guardian contacts
9	registration	Student-section enrollment transaction
10	attendance	Session attendance records
11	examination	Assessment records
12	grade	Trigger-generated grade result
13	fee_bill	Student fee invoices
14	payment	Fee payment transactions
15	user_account	Application authentication accounts

2.2 Keys and Integrity Constraints

The final SQL contains seven trigger definitions organized into five business-rule areas: registration validation, attendance validation, payment validation, fee-status recalculation, and automatic grade generation. This count distinguishes individual trigger definitions from the higher-level business-rule groups.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Business Rule Enforced
department	PK department_id; UNIQUE dept_code; NOT NULL dept_code, dept_name; CHECK established_year >= 1900; DEFAULT created_at	Course/programme/faculty relationships remain valid.
programme	PK programme_id; FK department_id; UNIQUE prog_code; CHECK duration_years 1-5; total_semesters 2-10	Programme must belong to an existing department.
faculty	PK faculty_id; FK department_id; UNIQUE employee_id/email; DEFAULT is_active TRUE	Faculty belongs to an existing department.
course	PK course_id; FK department_id; UNIQUE course_code; CHECK credits 1-8; DEFAULT course_level	Course catalogue remains unique and valid.
semester	PK semester_id; UNIQUE semester_code; CHECK start_date < end_date; DEFAULT is_active FALSE	Academic term dates remain ordered.
section	PK section_id; FK course_id/semester_id/faculty_id; UNIQUE course-semester-section; CHECK capacity > 0	Section references valid course/term/faculty and has positive capacity.
student	PK student_id; FK programme_id; UNIQUE reg_number/email; CHECK current_semester 1-10 and valid status; DEFAULT status Active	Student identity and programme relation remain unique.
guardian	PK guardian_id; FK student_id; NOT NULL contact fields; ON DELETE CASCADE	Guardian row cannot exist without a student.
registration	PK registration_id; FK student/course/semester/section; UNIQUE student-course-semester; CHECK status set; DEFAULT Enrolled	Duplicate enrollment is constrained and cross-row rules are trigger-enforced.
attendance	PK attendance_id; FK registration_id; UNIQUE registration_id + attendance_date; CHECK status in Present/Absent/Excused	One attendance status per registration per date.
examination	PK exam_id; FK registration_id; CHECK max_marks > 0 and marks_obtained <= max_marks; CHECK exam_type set	Invalid mark ranges are rejected.
grade	PK grade_id; UNIQUE exam_id; FK exam_id; NOT NULL calculated fields	One database-generated grade row per exam.

Table Name	Key / Constraint	Business Rule Enforced
fee_bill	PK bill_id; FK student_id/semester_id; UNIQUE bill_number; CHECK amount > 0; valid status	Bill must reference an existing student and semester.
payment	PK payment_id; FK bill_id; UNIQUE receipt_number; CHECK amount_paid > 0	Payment is linked to a valid bill; trigger prevents overpayment.
user_account	PK user_id; UNIQUE username/email; CHECK role; DEFAULT is_active TRUE	Login identity and role values remain constrained.

2.3 Functional Dependencies

- department_id → dept_code, dept_name, building, established_year, created_at
- department_id + dept_code → department attributes
- programme_id → department_id, prog_code, prog_name, degree_type, duration_years, total_semesters, created_at
- faculty_id → employee_id, department_id, first_name, last_name, email, phone, designation, joining_date, is_active, created_at
- course_id → department_id, course_code, course_name, credits, course_level, description, created_at
- semester_id → semester_code, semester_name, academic_year, start_date, end_date, is_active, created_at
- section_id → course_id, semester_id, faculty_id, section_name, capacity, room_no, created_at
- student_id → reg_number, programme_id, first_name, last_name, email, phone, dob, gender, admission_date, current_semester, status, created_at
- guardian_id → student_id, guardian_name, relationship, phone, email, address, created_at
- registration_id → student_id, course_id, semester_id, section_id, registration_date, status, created_at
- attendance_id → registration_id, attendance_date, status, remarks, recorded_at
- exam_id → registration_id, exam_type, exam_date, max_marks, marks_obtained, created_at
- grade_id → exam_id, percentage, letter_grade, grade_point, remarks, calculated_at
- bill_id → bill_number, student_id, semester_id, fee_type, amount, due_date, issue_date, status, created_at
- payment_id → receipt_number, bill_id, amount_paid, payment_date, payment_mode, transaction_ref, created_at
- user_id → username, password_hash, role, reference_id, email, is_active, last_login, created_at

For registration, the alternate key (student_id, course_id, semester_id) determines the registration row for the selected course and semester, while registration_id remains the surrogate primary key. These dependencies are used to justify decomposition into entity-specific relations.

2.4 Normalization up to Third Normal Form (3NF)

Table 7. Normalization Status up to 3NF

Relation	1NF	2NF	3NF	BCNF	Remarks / Decomposition
department	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
programme	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
faculty	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
course	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
semester	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
section	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
student	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
guardian	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
registration	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
attendance	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the

Relation	1NF	2NF	3NF	BCNF	Remarks / Decomposition
					appropriate determinant.
examination	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
grade	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
fee_bill	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
payment	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.
user_account	Yes	Yes	Yes	—	Entity relation decomposed so non-key facts are stored with the appropriate determinant.

The design reaches 3NF for the scope of the project. Repeated facts are separated across tables, many-to-many participation is resolved through registration, attendance and examination reference registration rather than copying student/course attributes, and payment is kept separate from the bill record. Foreign keys provide the structural references between parent and child relations [5]. BCNF is not claimed as a separate proof in this report.

2.5 Data Dictionary

DEPARTMENT

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
department	department_id	INT	PK, AUTO_INCREMENT	Unique department identifier
department	dept_code	VARCHAR(10)	UNIQUE, NOT NULL	Short department code
department	dept_name	VARCHAR(100)	NOT NULL	Department name
department	building	VARCHAR(50)	—	Building / academic block
department	established_year	INT	CHECK >= 1900	Department establishment year
department	created_at	TIMESTAMP	DEFAULT	Creation timestamp

PROGRAMME

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
programme	programme_id	INT	PK, AUTO_INCREMENT	Unique programme identifier

Table	Attribute	Data Type	Key / Constraint	Description
programme	department id	INT	FK, NOT NULL	Owning department
programme	prog code	VARCHAR(15)	UNIQUE, NOT NULL	Programme code
programme	prog name	VARCHAR(100)	NOT NULL	Programme name
programme	degree type	VARCHAR(20)	NOT NULL	Degree type
programme	duration years	INT	CHECK 1–5	Programme duration
programme	total semesters	INT	CHECK 2–10	Total semesters
programme	created at	TIMESTAMP	DEFAULT	Creation timestamp

FACULTY

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
faculty	faculty id	INT	PK, AUTO INCREMENT	Faculty identifier
faculty	employee id	VARCHAR(20)	UNIQUE, NOT NULL	Employee identifier
faculty	department id	INT	FK, NOT NULL	Department reference
faculty	first name	VARCHAR(50)	NOT NULL	First name
faculty	last name	VARCHAR(50)	NOT NULL	Last name
faculty	email	VARCHAR(100)	UNIQUE, NOT NULL	Faculty email
faculty	phone	VARCHAR(20)	—	Phone number
faculty	designation	VARCHAR(50)	NOT NULL	Academic designation
faculty	joining date	DATE	NOT NULL	Joining date
faculty	is active	BOOLEAN	DEFAULT TRUE	Active status
faculty	created at	TIMESTAMP	DEFAULT	Creation timestamp

COURSE

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
course	course id	INT	PK, AUTO INCREMENT	Course identifier
course	department id	INT	FK, NOT NULL	Owning department
course	course code	VARCHAR(15)	UNIQUE, NOT NULL	Course code
course	course name	VARCHAR(120)	NOT NULL	Course title
course	credits	INT	CHECK 1–8	Credit units
course	course level	VARCHAR(30)	DEFAULT	Course level
course	description	TEXT	—	Course description
course	created at	TIMESTAMP	DEFAULT	Creation timestamp

SEMESTER

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
semester	semester id	INT	PK, AUTO INCREMENT	Semester identifier
semester	semester code	VARCHAR(20)	UNIQUE, NOT NULL	Semester code
semester	semester name	VARCHAR(50)	NOT NULL	Semester label
semester	academic year	VARCHAR(15)	NOT NULL	Academic year
semester	start date	DATE	NOT NULL	Start date
semester	end date	DATE	NOT NULL	End date
semester	is active	BOOLEAN	DEFAULT FALSE	Active term flag
semester	created at	TIMESTAMP	DEFAULT	Creation timestamp

SECTION

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
section	section_id	INT	PK, AUTO INCREMENT	Section identifier
section	course_id	INT	FK, NOT NULL	Course reference
section	semester_id	INT	FK, NOT NULL	Semester reference
section	faculty_id	INT	FK, NULL	Assigned faculty
section	section_name	VARCHAR(30)	NOT NULL	Section/cohort name
section	capacity	INT	CHECK > 0, DEFAULT 30	Seat capacity
section	room_no	VARCHAR(20)	—	Room allocation
section	created_at	TIMESTAMP	DEFAULT	Creation timestamp

STUDENT

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
student	student_id	INT	PK, AUTO INCREMENT	Student identifier
student	reg_number	VARCHAR(20)	UNIQUE, NOT NULL	University registration number
student	programme_id	INT	FK, NOT NULL	Programme reference
student	first_name	VARCHAR(50)	NOT NULL	First name
student	last_name	VARCHAR(50)	NOT NULL	Last name
student	email	VARCHAR(100)	UNIQUE, NOT NULL	Student email
student	phone	VARCHAR(20)	—	Phone number
student	dob	DATE	NOT NULL	Date of birth
student	gender	VARCHAR(15)	NOT NULL	Gender
student	admission_date	DATE	NOT NULL	Admission date
student	current_semester	INT	CHECK 1–10, DEFAULT 1	Current semester
student	status	VARCHAR(20)	CHECK, DEFAULT Active	Academic status
student	created_at	TIMESTAMP	DEFAULT	Creation timestamp

GUARDIAN

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
guardian	guardian_id	INT	PK, AUTO INCREMENT	Guardian identifier
guardian	student_id	INT	FK, NOT NULL	Student reference
guardian	guardian_name	VARCHAR(100)	NOT NULL	Guardian name
guardian	relationship	VARCHAR(50)	NOT NULL	Relationship to student
guardian	phone	VARCHAR(20)	NOT NULL	Guardian phone
guardian	email	VARCHAR(100)	—	Guardian email
guardian	address	VARCHAR(255)	—	Guardian address
guardian	created_at	TIMESTAMP	DEFAULT	Creation timestamp

REGISTRATION

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
registration	registration_id	INT	PK, AUTO INCREMENT	Registration identifier
registration	student_id	INT	FK, NOT NULL	Student reference
registration	course_id	INT	FK, NOT NULL	Course reference
registration	semester_id	INT	FK, NOT NULL	Semester reference
registration	section_id	INT	FK, NOT NULL	Section reference
registration	registration_date	DATE	NOT NULL	Registration date
registration	status	VARCHAR(20)	CHECK, DEFAULT Enrolled	Registration status
registration	created_at	TIMESTAMP	DEFAULT	Creation timestamp

ATTENDANCE**Table 8. Data Dictionary**

Table	Attribute	Data Type	Key / Constraint	Description
attendance	attendance_id	INT	PK, AUTO INCREMENT	Attendance identifier
attendance	registration_id	INT	FK, NOT NULL	Registration reference
attendance	attendance_date	DATE	UNIQUE with registration_id, NOT NULL	Session date
attendance	status	VARCHAR(15)	CHECK, NOT NULL	Present / Absent / Excused
attendance	remarks	VARCHAR(255)	—	Session remarks
attendance	recorded_at	TIMESTAMP	DEFAULT	Recorded timestamp

EXAMINATION**Table 8. Data Dictionary**

Table	Attribute	Data Type	Key / Constraint	Description
examination	exam_id	INT	PK, AUTO INCREMENT	Examination identifier
examination	registration_id	INT	FK, NOT NULL	Registration reference
examination	exam_type	VARCHAR(30)	CHECK, NOT NULL	Assessment type
examination	exam_date	DATE	NOT NULL	Exam date
examination	max_marks	DECIMAL(5,2)	CHECK > 0, DEFAULT 100	Maximum marks
examination	marks_obtained	DECIMAL(5,2)	CHECK >= 0 and <= max	Marks obtained
examination	created_at	TIMESTAMP	DEFAULT	Creation timestamp

GRADE**Table 8. Data Dictionary**

Table	Attribute	Data Type	Key / Constraint	Description
grade	grade_id	INT	PK, AUTO INCREMENT	Grade identifier
grade	exam_id	INT	FK, UNIQUE	Exam reference
grade	percentage	DECIMAL(5,2)	NOT NULL	Calculated percentage
grade	letter_grade	VARCHAR(5)	NOT NULL	Calculated letter grade
grade	grade_point	DECIMAL(3,1)	NOT NULL	Calculated grade point
grade	remarks	VARCHAR(50)	—	Evaluation text
grade	calculated_at	TIMESTAMP	DEFAULT	Calculation timestamp

FEE_BILL**Table 8. Data Dictionary**

Table	Attribute	Data Type	Key / Constraint	Description
fee_bill	bill_id	INT	PK, AUTO INCREMENT	Bill identifier
fee_bill	bill_number	VARCHAR(30)	UNIQUE, NOT NULL	Bill reference number
fee_bill	student_id	INT	FK, NOT NULL	Student reference
fee_bill	semester_id	INT	FK, NOT NULL	Semester reference
fee_bill	fee_type	VARCHAR(50)	DEFAULT Tuition Fee, NOT NULL	Fee category
fee_bill	amount	DECIMAL(10,2)	CHECK > 0, NOT NULL	Amount due
fee_bill	due_date	DATE	NOT NULL	Due date
fee_bill	issue_date	DATE	NOT NULL	Issue date
fee_bill	status	VARCHAR(20)	CHECK, DEFAULT	Settlement status

Table	Attribute	Data Type	Key / Constraint	Description
			Unpaid	
fee_bill	created at	TIMESTAMP	DEFAULT	Creation timestamp

PAYMENT

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
payment	payment id	INT	PK, AUTO INCREMENT	Payment identifier
payment	receipt number	VARCHAR(30)	UNIQUE, NOT NULL	Receipt identifier
payment	bill id	INT	FK, NOT NULL	Fee bill reference
payment	amount paid	DECIMAL(10,2)	CHECK > 0, NOT NULL	Payment amount
payment	payment date	DATE	NOT NULL	Payment date
payment	payment mode	VARCHAR(30)	NOT NULL	Payment channel
payment	transaction ref	VARCHAR(50)	—	Transaction reference
payment	created at	TIMESTAMP	DEFAULT	Creation timestamp

USER_ACCOUNT

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
user_account	user id	INT	PK, AUTO INCREMENT	User identifier
user_account	username	VARCHAR(50)	UNIQUE, NOT NULL	Login username
user_account	password hash	VARCHAR(255)	NOT NULL	Bcrypt password hash
user_account	role	VARCHAR(20)	CHECK, NOT NULL	Admin / Faculty / Student
user_account	reference_id	INT	NULL	Optional academic entity reference
user_account	email	VARCHAR(100)	UNIQUE, NOT NULL	Account email
user_account	is active	BOOLEAN	DEFAULT TRUE	Account status
user_account	last login	TIMESTAMP	NULL	Last successful login
user_account	created at	TIMESTAMP	DEFAULT	Creation timestamp

2.6 Implementation of Logical Design: DDL and Sample Data

Table 9. Database Implementation Environment

Item	Details
DBMS	MySQL Server 8.0+
Storage Engine	InnoDB
Character Set	utf8mb4
Tables	15 relational tables
Seed Dataset	Fictional records for academic demonstration
Repository	https://github.com/404Vardan/dbms-pbl

Representative DDL excerpt:

```
CREATE TABLE course (
    course_id INT AUTO_INCREMENT PRIMARY KEY,
    department_id INT NOT NULL,
    course_code VARCHAR(15) NOT NULL UNIQUE,
    course_name VARCHAR(120) NOT NULL,
    credits INT NOT NULL CHECK (credits BETWEEN 1 AND 8),
    CONSTRAINT fk_course_dept
        FOREIGN KEY (department_id) REFERENCES department(department_id)
);
```

The seed script populates five departments, six programmes, twelve faculty records, twelve courses, three semester records, ten sections, twenty-five fictional students, eleven guardian records, fifteen registrations, seventeen attendance records, eight examinations, eight grades, seven fee bills, five payments and four demo user accounts. These figures describe the repository seed data rather than a live institutional database.

Representative sample records:

Representative seed data used for academic demonstration includes 5 departments, 6 programmes, 12 faculty members, 12 courses, 3 semesters, 10 sections, 25 students, 11 guardians, 15 registrations, 17 attendance records, 8 examinations, 8 grade rows, 7 fee bills, 5 payments and 4 demonstration user accounts. These are seed records, not live university records.

CHAPTER 3**APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN****3.1 Application Overview**

SCMS is implemented as a small Python application using Streamlit for the user interface and mysql-connector-python for database connectivity. Streamlit applications are Python scripts that render interactive widgets and can be organized into multiple pages [3], while Python provides the application runtime and standard programming environment [4]. In the SCMS prototype, the application calls MySQL directly through a utility layer; page modules do not maintain a separate copy of the academic or financial dataset.

The application starts at app.py, initializes session state, shows a login portal to unauthenticated users and exposes navigation to academic and database demonstration modules after authentication. Dashboard and reports query MySQL for current values, while transaction screens use parameterized statements and stored procedures to write data. Database-trigger errors are translated by the utility layer into readable application messages.

3.2 Development Environment and Database Connectivity**Table 10. Application Development Environment**

Item	Details
Programming Language	Python 3.11+
Framework / UI Toolkit	Streamlit 1.30+
DBMS	MySQL 8.0+
Database Connector	mysql-connector-python
Authentication	bcrypt + Streamlit session state
Configuration	python-dotenv and .env variables
Operating Environment	Local development / browser-based Streamlit client
Repository Link	https://github.com/404Vardan/dbms-pbl

3.3 Application Modules and Database Mapping**Table 11. Application Module-to-Database Mapping**

Module / Screen	Purpose	Tables Used	Main Operations
Login	Authenticate users and load	user_account	SELECT + UPDATE last_login

Module / Screen	Purpose	Tables Used	Main Operations
	role/session		
Dashboard	Live institutional metrics	department, programme, faculty, course, student, fee bill	SELECT / aggregates
Students	Search, view, admit and maintain student records	student, guardian, programme	SELECT, INSERT, UPDATE, procedure call
Departments / Programmes	Manage academic structure	department, programme	CRUD
Faculty	Faculty directory	faculty, department	CRUD / SELECT
Courses	Course catalogue	course, department	CRUD / SELECT
Semesters	Academic terms	semester	CRUD / SELECT
Sections	Sections and occupancy	section, course, semester, faculty, registration	CRUD / view query
Registration	Enrollment and trigger lab	registration, student, course, semester, section	INSERT, status update, trigger
Attendance	Record and summarize attendance	attendance, registration, student, course	INSERT, SELECT, trigger
Examinations	Record/update marks and verify grading	examination, grade, registration	procedure, UPDATE, trigger, views
Fees	Issue bills and record payments	fee bill, payment, student, semester	INSERT, SELECT, trigger
Reports	Academic, attendance, exam, fee and department reporting	8 SQL views + base joins	SELECT / filters

3.4 CRUD Operations

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Student admission, departments, programmes, faculty, courses, semesters, sections, registrations, attendance, exams, bills and payments	INSERT / procedure calls	Demonstrated through forms; database constraints apply.
Read / Search	Student directories, course lists, section data, dashboard and reports	SELECT, JOIN, GROUP BY	Loaded directly from MySQL / SQL views.
Update	Student details, attendance/exam marks, controlled statuses	UPDATE	Writes are parameterized; exam updates invoke grade recalculation.
Delete / Controlled Removal	Records with no dependent constraints may be deleted; registration/student state may also be changed through controlled status updates	DELETE / UPDATE	Foreign keys restrict invalid destructive changes; application presents error messages.

3.5 Business Rules, Validation and Error Handling

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Active student registration	MySQL BEFORE INSERT trigger checks student status	Registration rejected message when status is not Active.
Duplicate course-semester registration	UNIQUE constraint + trigger check	Duplicate active enrollment is rejected.
Section capacity	Trigger counts enrolled registrations before insert	Registration rejected when capacity is reached.
Attendance validity	Trigger blocks missing/dropped registrations and future dates	Readable database error is returned to the page.
Exam marks	CHECK marks >= 0 and <= max_marks	Invalid values rejected by database / form.
Grade calculation	AFTER INSERT / UPDATE trigger calculates percentage and grade	Grade row updated automatically.
Payment amount	CHECK > 0 + trigger checks outstanding balance	Overpayment is rejected.
Fee status	AFTER INSERT / DELETE trigger recalculates status	Bill status becomes Unpaid / Partially Paid / Paid.

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output	Purpose
Academic History	v_student_academic_history	JOIN across student, programme, registration, course, semester, examination and grade	Student-wise transcript-style history
Attendance Summary	v_attendance_summary	JOIN + conditional SUM/COUNT + percentage calculation	Session totals and attendance percentage
Examination Details	v_exam_details	JOIN exam records to trigger-generated grades	Marks and grades by assessment
Result Analysis	v_result_analysis	GROUP BY + AVG/MAX/MIN + pass percentage	Course-wise performance summary
Fee Dues	v_fee_dues	LEFT JOIN + SUM(payment)	Bill-wise paid amount and balance
Student Dues	v_student_dues	Aggregation over bills and payment totals	Student-level outstanding dues
Department Summary	v_department_summary	LEFT JOIN + COUNT DISTINCT	Department, faculty, programme, course and student counts

Report / Operation	SQL / Database Concept Used	Application Output	Purpose
Section Occupancy	v_section_occupancy	COUNT active enrollments against capacity	Seats available and occupancy percentage

3.7 Application Screens

Figures 2 to 6 were captured from the running Streamlit application connected to the MySQL database created from the repository scripts (schema.sql, seed.sql, triggers.sql, procedures.sql, views.sql). The screens show the authentication portal, the dashboard populated from SQL views, trigger-based rejections on registration and payment, and automatic grade generation after an examination mark is submitted through the stored procedure.

Student & College Management System
Woxsen University — Database Management Systems (DBMS) PBL Prototype

University Authentication Portal
Sign in to manage student admissions, section capacities, attendance compliance, examination marks, and fee billing.

PBL Evaluation Quick-Login:

- Admin: `admin / admin123` (Full university system privileges)
- Faculty: `faculty / faculty123` (Grading, attendance, courses)
- Student: `student / student123` (Vardan Desai transcript & dues)

Sign In

Username
Enter username (e.g. admin)

Password
Enter password

Log In

One-Click Demo Credentials:

[Fill Admin](#) [Fill Faculty](#) [Fill Student](#)

Figure 2. Login Screen

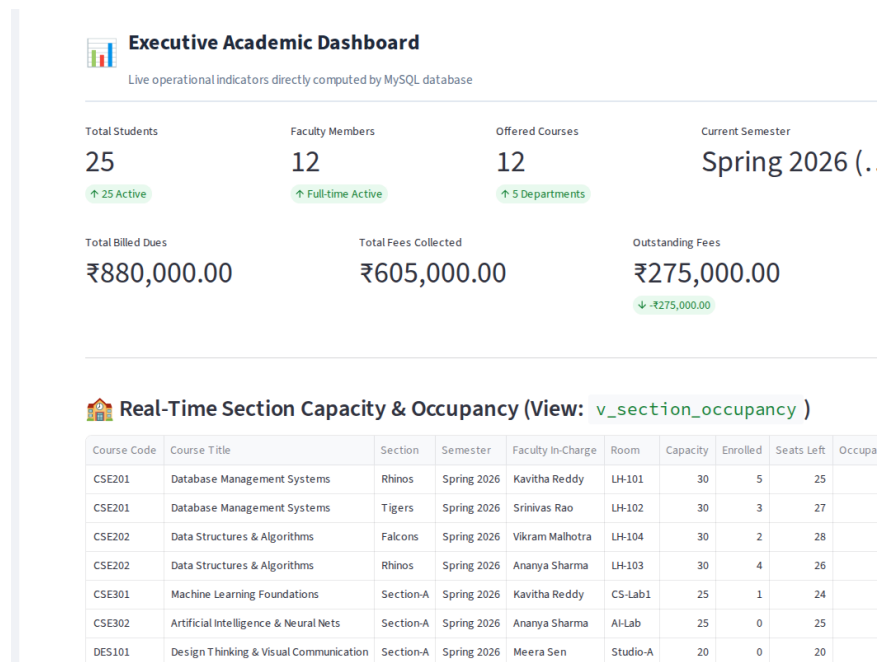


Figure 3. Dashboard and Live Database Metrics

 new course registration

 enrolled registrations

 test trigger validation

 **Live Trigger Validation Lab (PBL Viva Demo)**

Easily demonstrate DBMS trigger business rules live in front of the evaluator:

Test 1: Duplicate Registration

Attempts to register **Vardan Desai** for **CSE201** in **Spring 2026** again.

Run Duplicate Registration Test

 Trigger fired as expected:
Registration rejected: Student is already registered for this course in the selected semester.

Test 2: Section Capacity Exceeded

Tests trigger when a section's registered students reach capacity.

Run Section Capacity Test

 Trigger fired as expected:
Registration rejected: Section capacity has been reached.

(a) Test 1: duplicate registration rejected by `trg_registration_before_insert`

 new course registration

 enrolled registrations

 test trigger validation

 **Live Trigger Validation Lab (PBL Viva Demo)**

Easily demonstrate DBMS trigger business rules live in front of the evaluator:

Test 1: Duplicate Registration

Attempts to register **Vardan Desai** for **CSE201** in **Spring 2026** again.

Run Duplicate Registration Test

Test 2: Section Capacity Exceeded

Tests trigger when a section's registered students reach capacity.

Run Section Capacity Test

 Trigger fired as expected:
Registration rejected: Section capacity has been reached.

(b) Test 2: registration into a full section rejected by the capacity rule

Figure 4. Registration / Trigger Validation Screen



Examinations & Automated Database Grading

Evaluation engine powered by triggers and stored procedures



DBMS Trigger Demonstration — `trg_exam_after_insert` & `trg_exam_after_update`: Whenever marks are inserted or updated in MySQL:

- The database trigger automatically calculates the percentage.
- It computes the letter grade (A+, A, B+, B, C, D, F) and grade point (10.0 to 0.0).
- It inserts/updates the grade table directly in MySQL without relying on Python logic!



[Enter Exam Marks \(Procedure/Trigger\)](#)



[Update Marks & Verify Recalculation](#)



[Exam Grades \(View: `v\_exam\_details`\)](#)



[Result Analysis \(View: `v\_result\_analysis`\)](#)

Record Examination Result

Select Student & Course *

Reg #4 | 25WU0104001 - Aarav Gupta | CSE201 (Spring 2026)

Exam Type *

Mid-Term

Maximum Marks *

100.00

Examination Date *

2026 / 10 / 10

Marks Obtained *

92.50



Submit Marks via Stored Procedure (`sp_record_exam_result`)



SUCCESS: Exam result recorded and grade generated automatically by database trigger.



Database Trigger Computed Result:

- **Percentage:** 92.50%
- **Letter Grade:** A+
- **Grade Point:** 10.0 / 10.0

Figure 5. Examination and Automatic Grade Screen

Fee Billing & Financial Settlements

Tuition invoicing and transaction engine backed by integrity triggers

 **DBMS Trigger Demonstrations** — `trg_payment_before_insert` & `trg_payment_after_insert`:

1. **Overpayment Prevention:** Rejects any payment exceeding the outstanding balance on the bill (`amount_paid > balance`).
2. **Automated Status Recalculation:** Immediately updates the fee bill's status to **Paid**, **Partially Paid**, or **Unpaid** based on total payment sum!

 Outstanding Fee Bills (SQL View)
  Record Fee Payment
  Generate New Fee Bill
  Payment Transactions
  **Test Payment Triggers**

Live Trigger Validation Lab (PBL Viva Demo)

Test: Overpayment Beyond Outstanding Balance

Attempts to submit a payment exceeding the outstanding balance on a bill.

- E.g. If balance is ₹5,000, attempting to pay ₹100,000 will be caught by `trg_payment_before_insert`.

Run Overpayment Test

 **Trigger fired as expected:**

Payment rejected: Payment amount exceeds outstanding balance.

(a) Overpayment rejected by `trg_payment_before_insert`

Comprehensive Academic SQL Reports

Analytical business intelligence computed strictly via database views

 **DBMS Views Demonstration:** Every table and aggregation shown in this module is queried directly from predefined MySQL Views:

- `v_student_academic_history`
- `v_attendance_summary`
- `v_result_analysis` & `v_exam_details`
- `v_fee_dues` & `v_student_dues`
- `v_department_summary` & `v_section_occupancy` No business aggregations are calculated in application Python code.

 Academic History
  Attendance Summary
  Examination Analysis
  **Fee & Dues Ledger**
 Department Summary

Aggregated Student Dues (`v_student_dues`)

Reg Number	Student Name	Programme	Total Invoiced (₹)	Total Cleared (₹)	Outstanding Dues (₹)	Pend
25WU0104002	Aditi Verma	B.Tech Computer Science & Engineering (AI & ML)	150000	0	150000	
25WU0104008	Neha Bhatia	B.Tech Computer Science (Data Science)	140000	70000	70000	
25WU0104001	Aarav Gupta	B.Tech Computer Science & Engineering (AI & ML)	150000	100000	50000	
25WU0104029	Vardan Desai	B.Tech Computer Science & Engineering (AI & ML)	155000	150000	5000	
25WU0104023	Ayaan Ali	Bachelor of Design (Industrial & Product Design)	0	0	0	
25WU0104022	Zara Khan	Bachelor of Design (Industrial & Product Design)	0	0	0	
25WU0104021	Yash Singhanian	Bachelor of Business Administration (Finance & Analytics)	0	0	0	
25WU0104020	Varun Teja	Bachelor of Business Administration (Finance & Analytics)	0	0	0	
25WU0104019	Utkarsh Saxena	B.Tech Mechanical & Robotics Engineering	0	0	0	
25WU0104018	Tanvi Pandey	B.Tech Mechanical & Robotics Engineering	0	0	0	

(b) Student dues ledger read from the `v_student_dues` view

Figure 6. Fee and SQL Reports Screen

3.8 Testing and Results

All test cases were executed on 10-10-2026 against a database freshly created from the repository scripts (MySQL Server 8.0.46, InnoDB), with the application running on Python 3.12, Streamlit 1.65.0, mysql-connector-python 26.7.0 and bcrypt 5.0.0. Each case was run as SQL against the same objects the application calls, and the user-interface cases (login, trigger lab, grading, overpayment) were additionally exercised through the Streamlit pages shown in Figures 2-6. Expected and actual results are listed in Table 15; the raw command output is reproduced in Appendix C.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Login	Use valid demo credentials	Successful authentication and session initialized	Seeded accounts verified with bcrypt: correct passwords accepted, wrong password rejected; admin login opened the dashboard with role Admin (Figures 2-3).	PASS
TC-02	Student admission	Submit valid student + guardian data	Student and guardian inserted atomically	sp_admit_student returned student_id 26 with SUCCESS message and created the guardian row; a repeated reg_number returned ERROR with rollback (0 rows left).	PASS
TC-03	Duplicate registration	Register same student for same course and semester again	Database rejects duplicate enrollment	Rejected: "Registration rejected: Student is already registered for this course in the selected semester." (SQLSTATE 45000).	PASS
TC-04	Full section	Attempt registration after capacity is reached	Trigger rejects the transaction	First registration accepted (1 enrolled); second rejected: "Registration rejected: Section capacity has been reached." Enrolled count stayed 1.	PASS
TC-05	Future attendance	Insert attendance_date > CURRENT_DATE	Trigger rejects the transaction	Rejected: "Attendance rejected: Attendance date cannot be in the future." No future rows stored.	PASS
TC-06	Automatic grading	Insert 92.50/100 exam marks	Grade becomes A+ with 10.0 point	92.50/100 gave percentage 92.50, grade A+, grade point 10.0. Boundaries: 39.99 = F (0.0), 40.00 = D (5.0), 80.00 = A (9.0).	PASS
TC-07	Overpayment	Pay more than remaining bill balance	Trigger rejects payment	Bill of 5,000: pay 2,000 = Partially Paid; pay 4,000 rejected "exceeds outstanding balance"; pay 3,000 = Paid; deleting that payment = Partially Paid.	PASS
TC-08	Report accuracy	Update a source row and refresh report	View-backed output reflects the database state	v_fee_dues showed paid 2,000 / balance 3,000 (Partially Paid), then paid 5,000 / balance 0.00 (Paid) immediately after the new payment.	PASS
TC-09	Inactive student	Register a student whose status is Suspended	Trigger rejects registration	Rejected: "Registration rejected: Only Active students can register for courses."	PASS
TC-10	Dropped registration	Insert attendance for a Dropped registration	Trigger rejects the record	Rejected: "Attendance rejected: Cannot record attendance for a dropped registration." 0 rows stored.	PASS
TC-11	Grade recalculation	Update exam 8 marks from 42 to 95	Grade row recalculated automatically	Grade changed from D (5.0) to A+ (10.0) without any application code.	PASS
TC-12	Invalid marks	Insert marks 30 against max 25	CHECK constraint rejects row	Rejected: Check constraint chk_marks_bounds is violated (error 3819).	PASS
TC-13	Valid attendance	Insert Present for an enrolled registration dated in the past	Record accepted	Row inserted successfully; no trigger or constraint violation.	PASS

All 13 test cases passed. The rejection cases show that integrity is enforced inside MySQL through triggers, CHECK constraints and transactional stored procedures, so the same rules apply to any client that connects to the database, not only to the Streamlit pages.

CHAPTER 4

CONCLUSION

The Student & College Management System demonstrates how a normalized relational database can support a small but connected set of college administration workflows. The final design contains 15 tables connected through primary and foreign keys, supported by declarative constraints, seven trigger definitions grouped across five business-rule areas, two stored procedures and eight analytical views. The Python Streamlit application provides the interface through which these database structures can be inspected and exercised.

The main DBMS learning outcome is the separation of concerns between application interaction and data integrity. Registration, attendance, payment and grade rules are represented in MySQL so that the database remains the authoritative decision point for the stored records. The project therefore connects conceptual modelling, normalization, SQL implementation and application integration within one academic prototype. All thirteen test cases in Table 15 passed against the repository scripts on MySQL 8.0, and the five application screens in Figures 2-6 show the corresponding behavior in the running Streamlit prototype.

4.1 Limitations and Future Enhancements

- The current application is a small-scope academic prototype and is not integrated with live university ERP or identity systems.
- Role values are stored and displayed for demonstration; fine-grained per-page authorization can be expanded in a future version.
- Testing was functional and single-user on the seeded demonstration dataset; concurrent-load, performance and security penetration testing were outside the scope of this prototype.
- Future enhancements may include timetable integration, richer audit logging, more granular permissions, automated notifications and broader reporting workflows.

5. REFERENCES

[1] Oracle Corporation, “MySQL 8.0 Reference Manual,” MySQL. Accessed: Oct. 8, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>

[2] Oracle Corporation, “Using Triggers,” MySQL 8.0 Reference Manual. Accessed: Oct. 8, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/triggers.html>

[3] Streamlit, “Streamlit Documentation,” Streamlit. Accessed: Oct. 8, 2026. [Online]. Available: <https://docs.streamlit.io/>

[4] Python Software Foundation, “Python 3 Documentation,” Python. Accessed: Oct. 8, 2026. [Online]. Available: <https://docs.python.org/3/>

[5] Oracle Corporation, “FOREIGN KEY Constraints,” MySQL 8.0 Reference Manual. Accessed: Oct. 8, 2026. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html>

6. APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete SQL implementation is submitted as repository files (database/schema.sql, seed.sql, triggers.sql, procedures.sql and views.sql in <https://github.com/404Vardan/dbms-pbl>) and is executed in that order. The listings below reproduce the key objects exactly as they appear in the repository: the DDL of the three most constraint-heavy tables, the validation triggers, the automatic grading trigger, the transactional stored procedure and two reporting views.

A.1 Execution order

```
mysql -u root -p < database/schema.sql      -- 15 tables, constraints, indexes
mysql -u root -p scms_db < database/seed.sql -- fictional demonstration data
mysql -u root -p scms_db < database/triggers.sql -- 7 trigger definitions
mysql -u root -p scms_db < database/procedures.sql -- sp_admit_student, sp_record_exam_result
mysql -u root -p scms_db < database/views.sql -- 8 analytical views
```

A.2 DDL: registration, examination and payment

```
CREATE TABLE registration (
  registration_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  course_id INT NOT NULL,
  semester_id INT NOT NULL,
  section_id INT NOT NULL,
  registration_date DATE NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'Enrolled' CHECK (status IN ('Enrolled', 'Dropped',
    'Completed')),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT uq_student_course_semester UNIQUE (student_id, course_id, semester_id),
  CONSTRAINT fk_reg_student FOREIGN KEY (student_id)
    REFERENCES student(student_id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_reg_course FOREIGN KEY (course_id)
    REFERENCES course(course_id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_reg_sem FOREIGN KEY (semester_id)
    REFERENCES semester(semester_id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_reg_section FOREIGN KEY (section_id)
    REFERENCES section(section_id) ON DELETE RESTRICT ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE examination (
  exam_id INT AUTO_INCREMENT PRIMARY KEY,
  registration_id INT NOT NULL,
  exam_type VARCHAR(30) NOT NULL CHECK (exam_type IN ('Mid-Term', 'End-Term', 'Quiz',
    'Assignment', 'Lab-Exam')),
  exam_date DATE NOT NULL,
  max_marks DECIMAL(5,2) NOT NULL DEFAULT 100.00 CHECK (max_marks > 0),
  marks_obtained DECIMAL(5,2) NOT NULL CHECK (marks_obtained >= 0),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT chk_marks_bounds CHECK (marks_obtained <= max_marks),
  CONSTRAINT fk_exam_reg FOREIGN KEY (registration_id)
    REFERENCES registration(registration_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE payment (
  payment_id INT AUTO_INCREMENT PRIMARY KEY,
  receipt_number VARCHAR(30) NOT NULL UNIQUE,
  bill_id INT NOT NULL,
  amount_paid DECIMAL(10,2) NOT NULL CHECK (amount_paid > 0),
  payment_date DATE NOT NULL,
  payment_mode VARCHAR(30) NOT NULL,
  transaction_ref VARCHAR(50),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_pay_bill FOREIGN KEY (bill_id)
    REFERENCES fee_bill(bill_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

A.3 Validation triggers

```

CREATE TRIGGER trg_registration_before_insert
BEFORE INSERT ON registration
FOR EACH ROW
BEGIN
    DECLARE v_student_status VARCHAR(20);
    DECLARE v_capacity INT;
    DECLARE v_enrolled_count INT;
    DECLARE v_duplicate_count INT;

    -- 1. Check student existence and status
    SELECT status INTO v_student_status
    FROM student
    WHERE student_id = NEW.student_id;

    IF v_student_status IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Registration rejected: Student does not exist.';
    ELSEIF v_student_status <> 'Active' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Registration rejected: Only Active students can register for
        courses.';
    END IF;

    -- 2. Check for duplicate registration in this course and semester
    SELECT COUNT(*) INTO v_duplicate_count
    FROM registration
    WHERE student_id = NEW.student_id
        AND course_id = NEW.course_id
        AND semester_id = NEW.semester_id
        AND status <> 'Dropped';

    IF v_duplicate_count > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Registration rejected: Student is already registered for this course
        in the selected semester.';
    END IF;

    -- 3. Check section capacity
    SELECT capacity INTO v_capacity
    FROM section
    WHERE section_id = NEW.section_id;

    IF v_capacity IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Registration rejected: Specified section does not exist.';
    END IF;

    SELECT COUNT(*) INTO v_enrolled_count
    FROM registration
    WHERE section_id = NEW.section_id
        AND status = 'Enrolled';

    IF v_enrolled_count >= v_capacity THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Registration rejected: Section capacity has been reached.';
    END IF;
END

CREATE TRIGGER trg_attendance_before_insert
BEFORE INSERT ON attendance
FOR EACH ROW
BEGIN
    DECLARE v_reg_status VARCHAR(20);

    -- 1. Check registration status
    SELECT status INTO v_reg_status
    FROM registration
    WHERE registration_id = NEW.registration_id;

    IF v_reg_status IS NULL THEN

```

```

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Attendance rejected: Registration record not found.';
    ELSEIF v_reg_status = 'Dropped' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Attendance rejected: Cannot record attendance for a dropped
        registration.';
    END IF;

-- 2. Disallow future dates
IF NEW.attendance_date > CURRENT_DATE() THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Attendance rejected: Attendance date cannot be in the future.';
END IF;
END

CREATE TRIGGER trg_payment_before_insert
BEFORE INSERT ON payment
FOR EACH ROW
BEGIN
    DECLARE v_bill_amount DECIMAL(10,2);
    DECLARE v_already_paid DECIMAL(10,2);
    DECLARE v_balance DECIMAL(10,2);

    IF NEW.amount_paid <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Payment rejected: Payment amount must be greater than zero.';
    END IF;

    -- Fetch bill details
    SELECT amount INTO v_bill_amount
    FROM fee_bill
    WHERE bill_id = NEW.bill_id;

    IF v_bill_amount IS NULL THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Payment rejected: Associated fee bill not found.';
    END IF;

    -- Calculate total already paid
    SELECT IFNULL(SUM(amount_paid), 0.00) INTO v_already_paid
    FROM payment
    WHERE bill_id = NEW.bill_id;

    SET v_balance = v_bill_amount - v_already_paid;

    IF NEW.amount_paid > v_balance THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Payment rejected: Payment amount exceeds outstanding balance.';
    END IF;
END

```

A.4 Fee-status recalculation and automatic grading triggers

```

CREATE TRIGGER trg_payment_after_insert
AFTER INSERT ON payment
FOR EACH ROW
BEGIN
    DECLARE v_bill_amount DECIMAL(10,2);
    DECLARE v_total_paid DECIMAL(10,2);

    SELECT amount INTO v_bill_amount
    FROM fee_bill
    WHERE bill_id = NEW.bill_id;

    SELECT IFNULL(SUM(amount_paid), 0.00) INTO v_total_paid
    FROM payment
    WHERE bill_id = NEW.bill_id;

    IF v_total_paid >= v_bill_amount THEN
        UPDATE fee_bill SET status = 'Paid' WHERE bill_id = NEW.bill_id;
    ELSEIF v_total_paid > 0 THEN
        UPDATE fee_bill SET status = 'Partially Paid' WHERE bill_id = NEW.bill_id;
    ELSE

```

```

        UPDATE fee_bill SET status = 'Unpaid' WHERE bill_id = NEW.bill_id;
    END IF;
END

CREATE TRIGGER trg_exam_after_insert
AFTER INSERT ON examination
FOR EACH ROW
BEGIN
    DECLARE v_percentage DECIMAL(5,2);
    DECLARE v_letter VARCHAR(5);
    DECLARE v_gp DECIMAL(3,1);
    DECLARE v_rem VARCHAR(50);

    SET v_percentage = (NEW.marks_obtained / NEW.max_marks) * 100.00;

    IF v_percentage >= 90.00 THEN
        SET v_letter = 'A+';
        SET v_gp = 10.0;
        SET v_rem = 'Outstanding';
    ELSEIF v_percentage >= 80.00 THEN
        SET v_letter = 'A';
        SET v_gp = 9.0;
        SET v_rem = 'Excellent';
    ELSEIF v_percentage >= 70.00 THEN
        SET v_letter = 'B+';
        SET v_gp = 8.0;
        SET v_rem = 'Very Good';
    ELSEIF v_percentage >= 60.00 THEN
        SET v_letter = 'B';
        SET v_gp = 7.0;
        SET v_rem = 'Good';
    ELSEIF v_percentage >= 50.00 THEN
        SET v_letter = 'C';
        SET v_gp = 6.0;
        SET v_rem = 'Average';
    ELSEIF v_percentage >= 40.00 THEN
        SET v_letter = 'D';
        SET v_gp = 5.0;
        SET v_rem = 'Pass';
    ELSE
        SET v_letter = 'F';
        SET v_gp = 0.0;
        SET v_rem = 'Fail';
    END IF;

    INSERT INTO grade (exam_id, percentage, letter_grade, grade_point, remarks)
    VALUES (NEW.exam_id, v_percentage, v_letter, v_gp, v_rem)
    ON DUPLICATE KEY UPDATE
        percentage = v_percentage,
        letter_grade = v_letter,
        grade_point = v_gp,
        remarks = v_rem,
        calculated_at = CURRENT_TIMESTAMP;
END

```

The AFTER DELETE payment trigger (trg_payment_after_delete) repeats the same status logic using OLD.bill_id, and trg_exam_after_update repeats the grading logic for updated marks; both are in database/triggers.sql.

A.5 Stored procedure sp_admit_student (transactional)

```

CREATE PROCEDURE sp_admit_student(
    IN p_reg_number    VARCHAR(20),
    IN p_programme_id  INT,
    IN p_first_name    VARCHAR(50),
    IN p_last_name     VARCHAR(50),
    IN p_email         VARCHAR(100),
    IN p_phone         VARCHAR(20),
    IN p_dob           DATE,
    IN p_gender        VARCHAR(15),
    IN p_admission_date DATE,
    IN p_current_semester INT,
    IN p_guardian_name  VARCHAR(100),
    IN p_guardian_rel   VARCHAR(50),

```

```

    IN  p_guardian_phone  VARCHAR(20),
    IN  p_guardian_email  VARCHAR(100),
    IN  p_guardian_address VARCHAR(255),
    OUT p_student_id     INT,
    OUT p_status_message  VARCHAR(255)
)
proc_label: BEGIN
    DECLARE v_new_student_id INT DEFAULT NULL;
    DECLARE v_prog_exists INT DEFAULT 0;

    -- Exception Handler: Rollback transaction on any SQLEXCEPTION
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SET p_student_id = NULL;
        SET p_status_message = 'ERROR: Student admission failed. Transaction rolled back.';
    END;

    -- Validate Programme
    SELECT COUNT(*) INTO v_prog_exists
    FROM programme
    WHERE programme_id = p_programme_id;

    IF v_prog_exists = 0 THEN
        SET p_student_id = NULL;
        SET p_status_message = 'ERROR: Specified Programme ID does not exist.';
        LEAVE proc_label;
    END IF;

    -- Start Transaction
    START TRANSACTION;

    -- 1. Insert Student Record
    INSERT INTO student (
        reg_number, programme_id, first_name, last_name, email,
        phone, dob, gender, admission_date, current_semester, status
    ) VALUES (
        p_reg_number, p_programme_id, p_first_name, p_last_name, p_email,
        p_phone, p_dob, p_gender, p_admission_date, p_current_semester, 'Active'
    );

    SET v_new_student_id = LAST_INSERT_ID();

    -- 2. Insert Guardian Details if name provided
    IF p_guardian_name IS NOT NULL AND TRIM(p_guardian_name) <> '' THEN
        INSERT INTO guardian (
            student_id, guardian_name, relationship, phone, email, address
        ) VALUES (
            v_new_student_id, p_guardian_name, p_guardian_rel,
            p_guardian_phone, p_guardian_email, p_guardian_address
        );
    END IF;

    -- Commit on Success
    COMMIT;

    SET p_student_id = v_new_student_id;
    SET p_status_message = 'SUCCESS: Student admitted successfully with guardian record.';
END

```

A.6 Reporting views v_fee_dues and v_section_occupancy

```

CREATE VIEW v_fee_dues AS
SELECT
    b.bill_id,
    b.bill_number,
    s.reg_number,
    CONCAT(s.first_name, ' ', s.last_name) AS student_name,
    sem.semester_name,
    b.fee_type,
    b.amount AS bill_amount,
    IFNULL(SUM(p.amount_paid), 0.00) AS paid_amount,
    (b.amount - IFNULL(SUM(p.amount_paid), 0.00)) AS balance_due,
    b.due_date,

```

```

        b.status
FROM fee_bill b
JOIN student s ON b.student_id = s.student_id
JOIN semester sem ON b.semester_id = sem.semester_id
LEFT JOIN payment p ON b.bill_id = p.bill_id
GROUP BY
    b.bill_id, b.bill_number, s.reg_number, s.first_name,
    s.last_name, sem.semester_name, b.fee_type, b.amount, b.due_date, b.status;

CREATE VIEW v_section_occupancy AS
SELECT
    sec.section_id,
    sec.section_name,
    c.course_code,
    c.course_name,
    sem.semester_name,
    IFNULL(CONCAT(f.first_name, ' ', f.last_name), 'Unassigned') AS faculty_name,
    sec.room_no,
    sec.capacity,
    COUNT(CASE WHEN r.status = 'Enrolled' THEN 1 END) AS enrolled_count,
    (sec.capacity - COUNT(CASE WHEN r.status = 'Enrolled' THEN 1 END)) AS available_seats,
    ROUND((COUNT(CASE WHEN r.status = 'Enrolled' THEN 1 END) / sec.capacity) * 100.0, 1) AS
        occupancy_pct
FROM section sec
JOIN course c ON sec.course_id = c.course_id
JOIN semester sem ON sec.semester_id = sem.semester_id
LEFT JOIN faculty f ON sec.faculty_id = f.faculty_id
LEFT JOIN registration r ON sec.section_id = r.section_id
GROUP BY
    sec.section_id, sec.section_name, c.course_code, c.course_name,
    sem.semester_name, f.first_name, f.last_name, sec.room_no, sec.capacity;

```

APPENDIX B - APPLICATION SOURCE CODE

The application source is submitted in the repository. The primary entry point is app.py, with page modules under pages/ and database and authentication utilities under utils/. The application contains no business calculation or grading logic: it passes parameters to MySQL and renders the result.

B.1 Project structure

SCMS/	
app.py	login portal, session handling, navigation
database/	schema.sql, seed.sql, triggers.sql, procedures.sql, views.sql
pages/	dashboard, students, departments, programmes, faculty, courses, semesters, sections, registration, attendance, examinations, fees, reports
utils/db.py	connection, parameterized queries, procedure calls, error translation
utils/auth.py	bcrypt hashing, user verification, session state
utils/helpers.py	shared layout helpers and status badges
requirements.txt	streamlit, mysql-connector-python, bcrypt, python-dotenv

B.2 Database access layer (utils/db.py, excerpt)

```

def execute_update(query: str, params=None):
    """
    Execute a parameterized INSERT, UPDATE, or DELETE statement.
    Returns: (rowcount, last_insert_id)
    """
    conn = get_connection()
    cursor = conn.cursor()
    try:
        cursor.execute(query, params or ())
        rowcount = cursor.rowcount
        last_id = cursor.lastrowid
        conn.commit()
        return rowcount, last_id
    except mysql.connector.Error as err:
        conn.rollback()
        raise ValueError(parse_db_error(err)) from err

def call_procedure(proc_name: str, args=()):

```

```

"""
Call a MySQL stored procedure and return the modified argument list.
"""
conn = get_connection()
cursor = conn.cursor()
try:
    result_args = cursor.callproc(proc_name, list(args))
    conn.commit()
    return result_args
except mysql.connector.Error as err:
    conn.rollback()
    raise ValueError(parse_db_error(err)) from err

```

B.3 Authentication (utils/auth.py, excerpt)

```

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Verify a plaintext password against a bcrypt hash."""
    try:
        return bcrypt.checkpw(plain_password.encode("utf-8"), hashed_password.encode("utf-8"))
    except Exception:
        return False

def authenticate_user(username: str, password: str):
    """
    Look up user account in MySQL database, verify bcrypt hash,
    and update last_login timestamp.
    Returns user dict on success, None on failure.
    """
    query = """
        SELECT user_id, username, password_hash, role, reference_id, email, is_active
        FROM user_account
        WHERE username = %s
    """
    user = execute_query(query, (username.strip(),), fetch="one")
    if not user:
        return None

```

Trigger SIGNAL messages (MySQL error 1644) are returned unchanged by `parse_db_error` and displayed by the Streamlit pages, which is why the rejection messages in Figures 4 and 6 are the exact messages defined in `triggers.sql`.

APPENDIX C - ADDITIONAL TEST DATA AND OUTPUTS

This appendix records the raw output of the test run summarized in Table 15 (mysql command-line client, MySQL Server 8.0.46, 10-10-2026) and the size of the seed dataset.

C.1 Seed dataset after running `seed.sql`

```

department 5  programme 6  faculty 12  course 12  semester 3
section 10  student 25  guardian 11  registration 15  attendance 17
examination 8  grade 8  fee_bill 7  payment 5  user_account 4
Objects created: 15 tables, 7 triggers, 2 stored procedures, 8 views

```

C.2 Raw outputs of the test run

```

TC-02 CALL sp_admit_student('25WU0104099',1,'Test','Student',...,@sid,@msg)
student_id = 26 | SUCCESS: Student admitted successfully with guardian record.
(same reg_number again) student_id = NULL | ERROR: Student admission failed. Transaction
rolled back.
rows for the rejected email = 0

TC-03 INSERT INTO registration(...) VALUES (1,1,2,1,...)
ERROR 1644 (45000): Registration rejected: Student is already registered for this
course in the selected semester.

TC-04 capacity-1 section: first registration accepted (enrolled = 1)
ERROR 1644 (45000): Registration rejected: Section capacity has been reached.
enrolled after rejected attempt = 1

TC-05 INSERT INTO attendance(...) VALUES (1,'2027-12-31','Present')
ERROR 1644 (45000): Attendance rejected: Attendance date cannot be in the future.
future rows stored = 0

```

```

TC-06  CALL sp_record_exam_result(3,'Mid-Term','2026-09-15',100,92.50,...)
        percentage 92.50 | letter_grade A+ | grade_point 10.0
        boundary checks: 39.99 -> F 0.0 | 40.00 -> D 5.0 | 80.00 -> A 9.0

TC-07  bill 2 (5000.00): Unpaid -> pay 2000 -> Partially Paid
        pay 4000 -> ERROR 1644 (45000): Payment rejected: Payment amount exceeds outstanding
        balance.
        pay 3000 -> Paid | DELETE that payment -> Partially Paid

TC-08  v_fee_dues: bill_amount 5000.00 | paid 2000.00 | balance 3000.00 | Partially Paid
        after another 3000: paid 5000.00 | balance 0.00 | Paid

TC-09  ERROR 1644 (45000): Registration rejected: Only Active students can register for courses.
TC-10  ERROR 1644 (45000): Attendance rejected: Cannot record attendance for a dropped
        registration.
TC-11  exam 8: marks 42.00 -> D 5.0 ; after UPDATE marks 95 -> A+ 10.0
TC-12  ERROR 3819 (HY000): Check constraint 'chk_marks_bounds' is violated.

```

C.3 Authentication check

admin	(Admin)	correct password: True	wrong password: False	hash prefix \$2b\$12\$
faculty	(Faculty)	correct password: True	wrong password: False	hash prefix \$2b\$12\$
student	(Student)	correct password: True	wrong password: False	hash prefix \$2b\$12\$